> “Implement a PyTorch model class for predicting the next activity and remaining time in a business process. The model takes a tensor of shape `(batch, seq_len)` — a sequence of encoded integers representing activities. Inside: first, an embedding layer with `padding_idx=0` and `num_embeddings=vocab_size+1`. Then three parallel branches, each applying a sliding window of a fixed size (sizes 3, 5, and 7) along the time axis to extract local patterns using learnable filters. After each sliding window, apply batch normalization and a nonlinear activation. Then for each branch take the maximum value across the entire time axis (global max-pooling). Concatenate the outputs of all three branches. Apply dropout. Finally, two fully connected output layers: one returns logits of shape `(batch, num_activities)` for activity classification, the other returns `(batch, 1)` for remaining time regression. Constructor parameters: `vocab_size`, `embed_dim=64`, `num_filters=128`, `num_activities=10`, `dropout=0.1`. Name the class `MyktbekModel`.”

LISA Flash

Изучено create_tasks, 9 update_task

Now I'll implement the complete PyTorch model class with all the required components:

Изучено 9 update_task

Here is the complete implementation of the `MyktbekModel` class:

python Свернуть Запустить Копировать

```python
import torch
import torch.nn as nn

class MyktbekModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, num_filters=128, num_activities=10, dr
        super(MyktbekModel, self).__init__()

        # Embedding layer with padding_idx=0 and num_embeddings=vocab_size+1
        self.embedding = nn.Embedding(num_embeddings=vocab_size + 1, embedding_dim=embe

        # Three parallel branches with sliding window convolutions of sizes 3, 5, and 7
        # Input to conv: (batch, embed_dim, seq_len) after embedding
        # We use Conv1d with kernel_size for sliding window along time axis
        self.conv3 = nn.Conv1d(in_channels=embed_dim, out_channels=num_filters, kernel_
        self.conv5 = nn.Conv1d(in_channels=embed_dim, out_channels=num_filters, kernel_
        self.conv7 = nn.Conv1d(in_channels=embed_dim, out_channels=num_filters, kernel_
```

```
        # Batch normalization for each branch
        self.bn3 = nn.BatchNorm1d(num_filters)
        self.bn5 = nn.BatchNorm1d(num_filters)
        self.bn7 = nn.BatchNorm1d(num_filters)

        # Nonlinear activation (ReLU)
        self.relu = nn.ReLU()

        # Global max-pooling across time axis for each branch
        # After convolution, shape is (batch, num_filters, seq_len)
        # We'll use adaptive max pooling to get fixed size output regardless of sequenc
        self.pool = nn.AdaptiveMaxPool1d(1)

        # Dropout layer
        self.dropout = nn.Dropout(dropout)

        # Fully connected layers for outputs
        # After concatenation: 3 * num_filters features
        self.fc_activity = nn.Linear(3 * num_filters, num_activities)
        self.fc_time = nn.Linear(3 * num_filters, 1)

    def forward(self, x):
        """
        Args:
            x: tensor of shape (batch, seq_len) containing encoded activity integers

        Returns:
            activity_logits: tensor of shape (batch, num_activities) for activity class
            time_prediction: tensor of shape (batch, 1) for remaining time regression
        """
        # Embedding: (batch, seq_len) -> (batch, seq_len, embed_dim)
        embedded = self.embedding(x)

        # Transpose to (batch, embed_dim, seq_len) for Conv1d
        embedded = embedded.transpose(1, 2)

        # Branch 1: kernel size 3
        out3 = self.conv3(embedded)
        out3 = self.bn3(out3)
        out3 = self.relu(out3)
        out3 = self.pool(out3)  # (batch, num_filters, 1)
```

```
        out3 = out3.squeeze(-1)  # (batch, num_filters)

        # Branch 2: kernel size 5
        out5 = self.conv5(embedded)
        out5 = self.bn5(out5)
        out5 = self.relu(out5)
        out5 = self.pool(out5)  # (batch, num_filters, 1)
        out5 = out5.squeeze(-1)  # (batch, num_filters)

        # Branch 3: kernel size 7
        out7 = self.conv7(embedded)
        out7 = self.bn7(out7)
        out7 = self.relu(out7)
        out7 = self.pool(out7)  # (batch, num_filters, 1)
        out7 = out7.squeeze(-1)  # (batch, num_filters)

        # Concatenate all three branches: (batch, 3 * num_filters)
        concatenated = torch.cat([out3, out5, out7], dim=1)

        # Apply dropout
        dropped = self.dropout(concatenated)

        # Output layers
        activity_logits = self.fc_activity(dropped)  # (batch, num_activities)
        time_prediction = self.fc_time(dropped)  # (batch, 1)

        return activity_logits, time_prediction
```

This implementation includes:

1. **Embedding Layer**: Uses `nn.Embedding` with `padding_idx=0` and `num_embeddings=vocab_size+1` to handle the input sequences.
2. **Three Parallel Branches**: Each branch applies a 1D convolution with kernel sizes 3, 5, and 7 respectively to extract local patterns along the time axis. The padding ensures the output sequence length matches the input.
3. **Batch Normalization and ReLU**: Applied after each convolution in every branch.
4. **Global Max-Pooling**: Uses `AdaptiveMaxPool1d(1)` to reduce each branch's output to a single value per filter across the entire time axis.
5. **Concatenation**: Combines the outputs from all three branches into a single feature vector of size `3 * num_filters`.
6. **Dropout**: Applied to the concatenated features before the fully connected layers.

7. **Output Layers**:
    - `fc_activity` : Returns logits of shape `(batch, num_activities)` for activity classification
    - `fc_time` : Returns predictions of shape `(batch, 1)` for remaining time regression

The model can be instantiated with:

python Свернуть Запустить Копировать

```
model = MyktbekModel(vocab_size=100, embed_dim=64, num_filters=128, num_activities=10,
```

Implement a training function `train_myktbek_model` in PyTorch. It takes: `model` `(x, y_activity, y_time)` `num_epochs` `learning_rate` `device` model to the device, use Adam optimizer. Use CrossEntropyLoss for the activity prediction (logits vs integer class indices) and MSELoss for the remaining time prediction. In each epoch, iterate over all batches: move tensors to device, zero gradients, do a forward pass, compute both losses, sum them, call backward, step the optimizer. If `y_time` has shape `(batch,)` , unsqueeze it to `(batch, 1)` before computing MSELoss. Track the average combined loss per epoch and print it. Return a list of per-epoch average losses.

LISA Flash

Изучено create_tasks, 8 update_task

Here is the complete implementation of the `train_myktbek_model` function:

python Свернуть Запустить Копировать

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

def train_myktbek_model(model, train_loader, num_epochs, learning_rate, device):
    """
    Trains the MyktbekModel for activity prediction and remaining time regression.

    Args:
        model (nn.Module): The PyTorch model to train.
        train_loader (DataLoader): DataLoader yielding batches of (x, y_activity, y_tim
        num_epochs (int): Number of training epochs.
        learning_rate (float): Learning rate for the optimizer.
```

```
    device (str): Device to train on (e.g., 'cpu' or 'cuda').

Returns:
    list: Average combined loss per epoch.
"""
# Move model to device
model.to(device)

# Initialize Adam optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

# Define loss functions
criterion_activity = nn.CrossEntropyLoss()
criterion_time = nn.MSELoss()

# List to store average losses per epoch
epoch_losses = []

for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    num_batches = 0

    for batch_x, batch_y_activity, batch_y_time in train_loader:
        # Move tensors to device
        batch_x = batch_x.to(device)
        batch_y_activity = batch_y_activity.to(device)
        batch_y_time = batch_y_time.to(device)

        # Handle y_time shape: if (batch,), unsqueeze to (batch, 1)
        if batch_y_time.dim() == 1:
            batch_y_time = batch_y_time.unsqueeze(1)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        activity_logits, time_prediction = model(batch_x)

        # Compute losses
        loss_activity = criterion_activity(activity_logits, batch_y_activity)
```

```
            loss_time = criterion_time(time_prediction, batch_y_time)

            # Sum losses
            total_batch_loss = loss_activity + loss_time

            # Backward pass and optimize
            total_batch_loss.backward()
            optimizer.step()

            # Accumulate loss
            total_loss += total_batch_loss.item()
            num_batches += 1

        # Calculate average loss for the epoch
        avg_loss = total_loss / num_batches
        epoch_losses.append(avg_loss)

        # Print average loss
        print(f"Epoch {epoch + 1}/{num_epochs}, Average Loss: {avg_loss:.4f}")

    return epoch_losses
```

Key Features:

1. **Device Handling**: Moves the model and all input tensors to the specified device.
2. **Optimizer**: Uses Adam with the provided learning rate.
3. **Loss Functions**:
   - `CrossEntropyLoss` for activity classification (expects logits and integer class indices).
   - `MSELoss` for time regression.
4. **Shape Adjustment**: Automatically unsqueezes `y_time` from `(batch,)` to `(batch, 1)` if needed.
5. **Training Loop**: Iterates over epochs and batches, performs forward/backward passes, and accumulates losses.
6. **Tracking**: Stores and prints the average combined loss per epoch.
7. **Return Value**: Returns a list of average losses for each epoch.

Usage Example:

python ⌃ Свернуть Запустить Копировать

```
# Assuming you have a model, train_loader, and device set up
losses = train_myktbek_model(
    model=model,
```

```
    train_loader=train_loader,
    num_epochs=10,
    learning_rate=0.001,
    device='cuda'
)
```

I have a Python function `run_single_dataset` in `experiments/run_experiments.py` that already trains and evaluates a `ProcessTransformer` model. I need to add a new block inside the same function, right after the ProcessTransformer block (Step 3), that trains and evaluates a second model called `MyktbekModel`. The new block should follow exactly this structure: (1) instantiate `MyktbekModel` with `vocab_size=len(vocab)`, `embed_dim=64`, `num_filters=128`, `num_activities=len(vocab)`, `dropout=0.1` — model is already imported; (2) create a `create_data_loader(train_df, encoded_train, (train_loader, mean_val,` `train_myktbek_model(model, train_loader, num_epochs=num_epochs, learning_rate=1e-3, device=device)` — already imported; (4) run inference on the test set using `encoded_test` in batches of 64 with `torch.no_grad()`, convert activity indices back to names using `idx_to_act`, denormalize time predictions using `val * std_val + mean_val`; (5) call `evaluate_by_prefix_length(test_df, activity_preds, time_preds)`; (6) call `save_results(eval_df, 'myktbek_cnn', dataset_name, output_dir)`. Return only this new block of code, not the full function.

LISA Flash

python Свернуть Запустить Копировать

```
    # Step 4: Train and evaluate MyktbekModel
    print(f"\n--- Training MyktbekModel on {dataset_name} ---")

    # (1) Instantiate MyktbekModel
    myktbek_model = MyktbekModel(
        vocab_size=len(vocab),
        embed_dim=64,
        num_filters=128,
        num_activities=len(vocab),
        dropout=0.1
    )
```

```python
# (2) Create DataLoader for training
train_loader, mean_val, std_val = create_data_loader(
    train_df, encoded_train, vocab, batch_size=32, shuffle=True
)

# (3) Train the model
myktbek_losses = train_myktbek_model(
    myktbek_model,
    train_loader,
    num_epochs=num_epochs,
    learning_rate=1e-3,
    device=device
)

# (4) Run inference on test set
myktbek_model.eval()
activity_preds = []
time_preds = []

with torch.no_grad():
    # Process in batches of 64
    for i in range(0, len(encoded_test), 64):
        batch_encoded = encoded_test[i:i+64]
        batch_tensor = torch.tensor(batch_encoded, dtype=torch.long).to(device)

        act_logits, time_out = myktbek_model(batch_tensor)

        # Get predicted activity indices
        _, pred_indices = torch.max(act_logits, dim=1)
        activity_preds.extend(pred_indices.cpu().numpy())

        # Denormalize time predictions
        denorm_time = (time_out.cpu().numpy() * std_val) + mean_val
        time_preds.extend(denorm_time.flatten())

# Map activity indices back to names
activity_preds_names = [idx_to_act[idx] for idx in activity_preds]

# (5) Evaluate by prefix length
```

```
eval_df = evaluate_by_prefix_length(test_df, activity_preds_names, time_preds)

# (6) Save results
save_results(eval_df, 'myktbek_cnn', dataset_name, output_dir)

print(f"MyktbekModel evaluation complete for {dataset_name}.")
```

I have a Python script `experiments/run_experiments.py`. At the top of the file, after the existing imports, I need to add two new import lines to import `MyktbekModel` from `src.myktybek_model` and `train_myktbek_model` from `src.myktybek_train`. Write those two import lines.
Also, I have this inference code that has a bug — `encoded_test[i:i+64]` is already a PyTorch tensor (result of `encode_and_pad`), so wrapping it in `torch.tensor()` is wrong. Also, `pred_indices.cpu().numpy()` returns numpy integers which may not match dictionary keys. Fix both issues: use `.to(device)` directly on the slice, and convert predicted indices to Python int with `.tolist()` before mapping through `idx_to_act`. Return the corrected inference block only (the `with torch.no_grad()` part and the activity name mapping line).

LISA Flash

Here are the two import lines to add at the top of `experiments/run_experiments.py`:

python ⌃ Свернуть Запустить Копировать

```
from src.myktybek_model import MyktbekModel
from src.myktybek_train import train_myktbek_model
```

And here is the corrected inference block:

python ⌃ Свернуть Запустить Копировать

```
with torch.no_grad():
    # Process in batches of 64
    for i in range(0, len(encoded_test), 64):
        batch_encoded = encoded_test[i:i+64]
        # Ensure batch_encoded is a tensor and move to device directly
        batch_tensor = batch_encoded.to(device) if isinstance(batch_encoded, torch.

        act_logits, time_out = myktbek_model(batch_tensor)

        # Get predicted activity indices
```

```
            _, pred_indices = torch.max(act_logits, dim=1)

            # Convert to Python list of ints for safe dictionary lookup
            activity_preds.extend(pred_indices.tolist())

            # Denormalize time predictions
            denorm_time = (time_out.cpu().numpy() * std_val) + mean_val
            time_preds.extend(denorm_time.flatten())

    # Map activity indices back to names using Python integers
    activity_preds_names = [idx_to_act[int(idx)] for idx in activity_preds]
```

Note: The check `isinstance(batch_encoded, torch.Tensor)` was added to handle the case where `encoded_test` might be a list or numpy array in some contexts, but if you are certain it is always a tensor, you can simplify the line to just `batch_tensor = encoded_test[i:i+64].to(device)`.